

Rechnergestützte Numerik und Simulation

(am Beispiel von Matlab/Simulink)

Numerik 4:

Dynamische Systeme

Inhalt

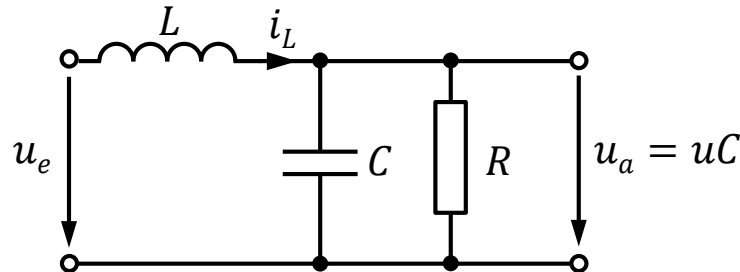
4. Dynamische Systeme

- 4.1 Beschreibung dynamischer Systeme
- 4.2 Plots für dynamische Systeme
- 4.3 Lösung dynamischer Systeme

4. Dynamische Systeme – 4.1 Beschreibung dyn. Sys.

Dynamische Systeme:

Folgender R-L-C-Filter soll untersucht und simuliert werden:



Der Filter stellt ein lineares zeitinvariantes System (LTI-System) dar und kann durch eine gewöhnliche Differenzialgleichung (ODE) zweiter Ordnung mathematisch beschrieben werden. Zur Darstellung dieser Gleichungen gibt es verschiedene Möglichkeiten, die mit den folgenden Befehlen realisiert werden können:

<code>sys = ss (...)</code>	Erstellt ein Zustandsraummodell
<code>sys = tf (...)</code>	Erstellt eine Übertragungsfunktion im Bildbereich in Polynom-Darstellung
<code>sys = zpk (...)</code>	Erstellt eine Übertragungsfunktion im Bildbereich in Pol-/Nullstellen-Darstellung.

Bei allen Befehlen wird ein LTI-Objekt angelegt, wodurch es ermöglicht wird bestehende Systeme in jede der anderen Darstellungen umzuwandeln und mehrere Systeme mit unterschiedlicher Darstellung miteinander zu verknüpfen.

4. Dynamische Systeme – 4.1 Beschreibung dyn. Sys

State-Space-Befehl:

Die Funktion `ss` erstellt ein kontinuierliches oder diskretes Zustandsraummodell (ZRM) und wandelt Systeme in anderen Darstellungen in die Zustandsraumdarstellung um.

Kontinuierliches ZRM:

$$\dot{x} = A \cdot x + B \cdot u$$
$$v = C \cdot x + D \cdot u$$

`sys_ss=ss(A,B,C,D)`

A, B, C, D : Kontinuierliche Systemmatrizen des Zustandsraummodells.

Diskretes ZRM:

$$x_{k+1} = \Phi \cdot x_k + H \cdot u_k$$
$$v = C \cdot x_k + D \cdot u_k$$

`sys_ss=ss(Φ,H,C,D,Ts)`

Φ, H, C, D : Diskrete Systemmatrizen des Zustandsraummodells.
 T_s : Abtastzeit des diskreten Systems.

Umwandlung:

`sys_ss=ss(sys)`

`sys`: Objekt eines Systems in einer anderen Darstellung.

4. Dynamische Systeme – 4.1 Beschreibung dyn. Sys

Beispiel: ss (...)

```

%% Parameter des RLC Netzwerks
L=10e-3;
C=500e-6;
R=10;
ue = 24;

%% Systemmatrizen des
kontinuierlichen Systems
A = [ 0 -1/L ; 1/C -1/(C*R) ];
B = [ 1/L ; 0 ];
C = [ 1 0 ];
D = [ 0 ];

%% Zustandsraummodell
zrm = ss(A,B,C,D);
  
```

Zustandsraumdarstellung des R-L-C-Filters:

$$\frac{d}{dt} \begin{bmatrix} i_L \\ u_C \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & -\frac{1}{L} \\ \frac{1}{C} & -\frac{1}{RC} \end{bmatrix}}_{\mathbf{A}} \cdot \begin{bmatrix} i_L \\ u_C \end{bmatrix} + \underbrace{\begin{bmatrix} 1/L \\ 0 \end{bmatrix}}_{\mathbf{B}} \cdot u_e$$

$$u_a = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{\mathbf{C}} \cdot \begin{bmatrix} i_L \\ u_C \end{bmatrix} + \underbrace{\begin{bmatrix} 0 \end{bmatrix}}_{\mathbf{D}} \cdot u_e$$

4. Dynamische Systeme – 4.1 Beschreibung dyn. Sys

tf-Befehl:

Die Funktion `tf` erstellt eine kontinuierliche oder diskrete Übertragungsfunktion und wandelt Systeme in anderer Darstellung in eine Übertragungsfunktion um.

Kontinuierliche Übertragungsfunktion:

`sys_tf=tf(num,den)`

$$G(s) = \frac{v}{u} = \frac{b_m \cdot s^m + \dots + b_2 \cdot s^2 + b_1 \cdot s + b_0}{a_n \cdot s^n + \dots + a_2 \cdot s^2 + a_1 \cdot s + a_0}$$

`num`: Zählerpolynom der Übertragungsfunktion (siehe: Kapitel 5.1).
`den`: Nennerpolynom der Übertragungsfunktion.

Diskrete Übertragungsfunktion:

`sys_tf=tf(num,den,Ts)`

$$G(z) = \frac{v}{u} = \frac{b_m \cdot z^m + \dots + b_2 \cdot z^2 + b_1 \cdot z + b_0}{a_n \cdot z^n + \dots + a_2 \cdot z^2 + a_1 \cdot z + a_0}$$

`num, den`: Zähler-/Nennerpolynom der Übertragungsfunktion (siehe: Kapitel 5.1).
`Ts`: Abtastzeit des diskreten Systems.

Umwandlung:

`sys_tf=tf(sys)`

`sys`: Objekt eines Systems in einer anderen Darstellung.

4. Dynamische Systeme – 4.1 Beschreibung dyn. Sys

Beispiel: `tf(...)`

Aus der Zustandsraumdarstellung des R-L-C-Filters wird die äquivalente Übertragungsfunktion bestimmt:

```
%% Parameter des RLC Netzwerks
L=10e-3;
C=500e-6;
R=10;
ue = 24;

%% Systemmatrizen des kontinuierlichen Systems
A = [ 0 -1/L ; 1/C -1/(C*R) ];
B = [ 1/L ; 0 ];
C = [1 0];
D = [0];

%% Zustandsraummodell
zrm = ss(A,B,C,D);

%% Umwandlung in Übertragungsfunktion
ueFkt = tf(zrm)
```

```
>>
Transfer function:
      100 s + 20000
-----
s^2 + 200 s + 2e005
```

4. Dynamische Systeme – 4.1 Beschreibung dyn. Sys

zpk-Befehl:

Die Funktion `zpk` erstellt eine kontinuierliche oder diskrete Übertragungsfunktion in Pol-/Nullstellen-Darstellung und wandelt Systeme in anderer Darstellung entsprechend um.

Kontinuierliche Übertragungsfunktion:

$$G(s) = \frac{v}{u} = k \cdot \frac{(s - n_m) \cdot \dots \cdot (s - n_2)(s - n_1)}{(s - p_n) \cdot \dots \cdot (s - p_2)(s - p_1)}$$

`sys_zpk=zpk(z,p,k)`

- z: Vektor mit den Nullstellen der Übertragungsfunktion
- p: Vektor mit den Polstellen der Übertragungsfunktion
- k: Verstärkung der Übertragungsfunktion

Diskrete Übertragungsfunktion:

$$G(z) = \frac{v}{u} = k \cdot \frac{(z - n_m) \cdot \dots \cdot (z - n_2)(z - n_1)}{(z - p_n) \cdot \dots \cdot (z - p_2)(z - p_1)}$$

`sys_zpk=zpk(z,p,k,Ts)`

- T_s: Abtastzeit des diskreten Systems.

Umwandlung:

`sys_zpk=zpk(sys)`

- sys: Objekt eines Systems in einer anderen Darstellung.

4. Dynamische Systeme – 4.1 Beschreibung dyn. Sys

Beispiel: `zpk(...)`

Aus der Zustandsraumdarstellung des R-L-C-Filters wird die äquivalente Übertragungsfunktion in Pol-Nullstellen-Darstellung bestimmt :

```
%% Parameter des RLC Netzwerks
L=10e-3;
C=500e-6;
R=10;
ue = 24;

%% Systemmatrizen des kontinuierlichen Systems
A = [ 0 -1/L ; 1/C -1/(C*R) ];
B = [ 1/L ; 0 ];
C = [1 0];
D = [0];

%% Zustandsraummodell
zrm = ss(A,B,C,D);

%% Umwandlung in Übertragungsfunktion
ueFkt = zpk(zrm)
```

```
>>
Zero/pole/gain:
      100 (s+200)
-----
(s^2 + 200s + 2e005)
```

Komplexe Pol-/Nullstellen
werden in dieser Darstellung
nicht angezeigt.

4. Dynamische Systeme – 4.1 Beschreibung dyn. Sys

Ist ein System gegeben, so können für jede beliebige Darstellung die charakteristischen Informationen extrahiert werden:

<code>[A,B,C,D]=ssdata(sys)</code>	Gibt die Systemmatrizen zurück.
<code>[num,den]=tfdata(sys)</code>	Gibt die Koeffizienten der Zähler- und Nennerpolynome zurück.
<code>[z,p,k] = zpkdata(sys)</code>	Gibt die Pol-, Nullstellen und statische Verstärkung zurück.

Hinweis: Die Ergebnisse der Funktionen werden zum Teil in einem Cell-Array zurückgegeben. Für den Zugriff aus solche Arrays muss `{ }` verwendet werden.

Beispiel: `tfdata(...)`

Es wird die Zustandsraumdarstellung des R-L-C-Filters verwendet, um die Koeffizienten der äquivalenten Übertragungsfunktion zu bestimmen.

```
>> [num,den]=tfdata(zrm);      >> denPoly =
>> denPoly = den{1};
                                1.0e+004 *
                                0.0001    0.0200    20.000
```

4. Dynamische Systeme – 4.1 Beschreibung dyn. Sys

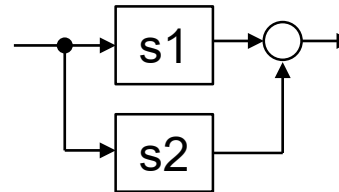
Verknüpfung von LTI-Objekten:

Auf LTI-Objekte können im Sinne der Wirkungsplanalgebra alle arithmetischen Standardoperationen angewendet werden:

Beispiele:

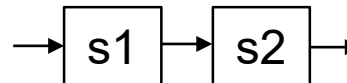
1. Parallelschaltung:

```
>> s = s1 + s2;
```



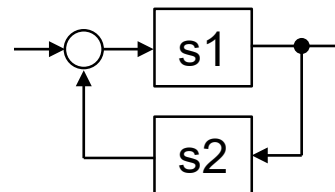
2. Serienschaltung:

```
>> s = s1 * s2;
```



3. Rückkuppelung:

```
>> s = s1 / (1-s1*s2);
```



Dabei spielt es keine Rolle, ob die LTI-Objekte in Zustandsdarstellung (ss), als Übertragungsfunktion (tf) oder in Pol-Nullstellendarstellung (zpk) angelegt worden sind.

4. Dynamische Systeme – 4.1 Beschreibung dyn. Sys

c2d-Befehl:

Der Befehl `c2d` überführt ein kontinuierliches System in ein diskretes System.

```
sysd = c2d(sys, Ts, 'method')
```

`sys`: Objekt eines Systems in einer der zuvor vorgestellten Darstellungen.
`Ts`: Abtastzeit des diskreten Systems.
`method`: Diskretisierungsmethoden
 ('zoh', 'foh', 'imp', 'tustin', 'prewarp', 'matched')

Beschreibung:

'zoh'	Zero-Order-Hold
'foh'	First-Order-Hold
'imp'	Impuls invariante Diskretisierung
'tustin'	Bilineare Approximation
'prewarp'	Bilineare Approximation mit pre-warping
'matched'	Matched pole-zero method

4. Dynamische Systeme – 4.1 Beschreibung dyn. Sys

Beispiel: `c2d(...)`

Es wird die Übertragungsfunktion des R-L-C-Filters gebildet und diskretisiert:

```
%%  
%% Umwandlung in Übertragungsfunktion  
ueFkt = tf(zrm)  
  
%% Diskretisierung  
dUeFkt = c2d(ueFkt, 8e-4);
```

```
>> dUeFkt
```

Transfer function:

```
0.09686 z - 0.07903  
-----  
z^2 - 1.64 z + 0.8187
```

Sampling time: 0.001

4. Dynamische Systeme – 4.2 Plots für dyn. Sys.

bode-Befehl:

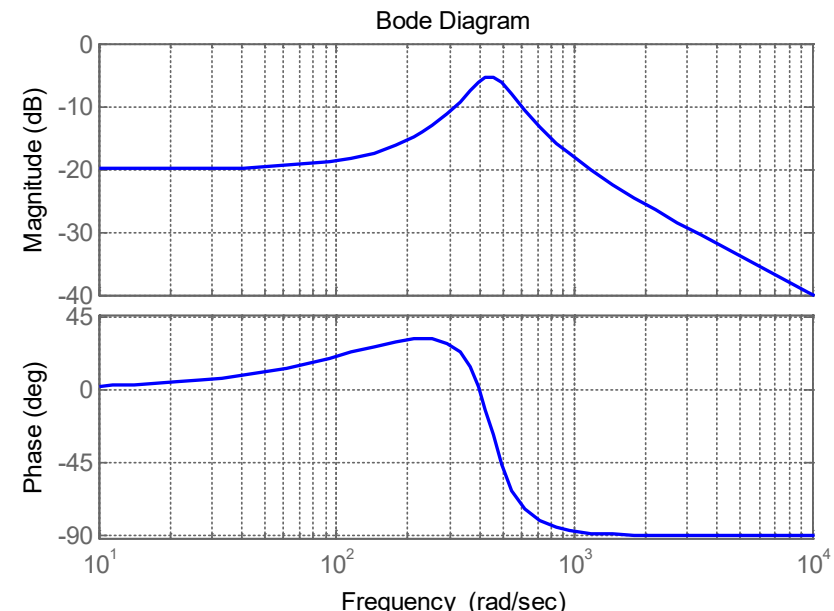
Mit der `bode`-Funktion kann der Amplituden- und Frequenzgang eines oder mehrerer LTI-Systeme angezeigt werden:

```
bode(sys1,...,sysn,w)
```

`sys1,...,sysn`: n LTI-Systeme in beliebiger Darstellung (`tf(...)`, `ss(...)`, `zpk(...)`)
`w`: Vektor der den darzustellenden Frequenzbereich definiert.

Beispiel: Frequenzgang des R-L-C-Filters

```
⋮  
%% Zustandsraummodell  
zrm= ss(A,B,C,D)  
  
%% Bode-Diagramm  
bode(zrm);  
grid on;
```



4. Dynamische Systeme – 4.2 Plots für dyn. Sys.

pzmap–**Befehl:**

Die pzmap–Funktion zeichnet den Pol-Nullstellenplan eines oder mehrerer LTI-Systeme:

```
pzmap(sys1,...,sysn)
```

sys1,...,sysn: n LTI-Systeme in beliebiger Darstellung ($\text{tf}(\dots)$, $\text{ss}(\dots)$, $\text{zpk}(\dots)$)

Die Nullstellen werden als Kreise und die Polstellen als Kreuze in dem Figure dargestellt.

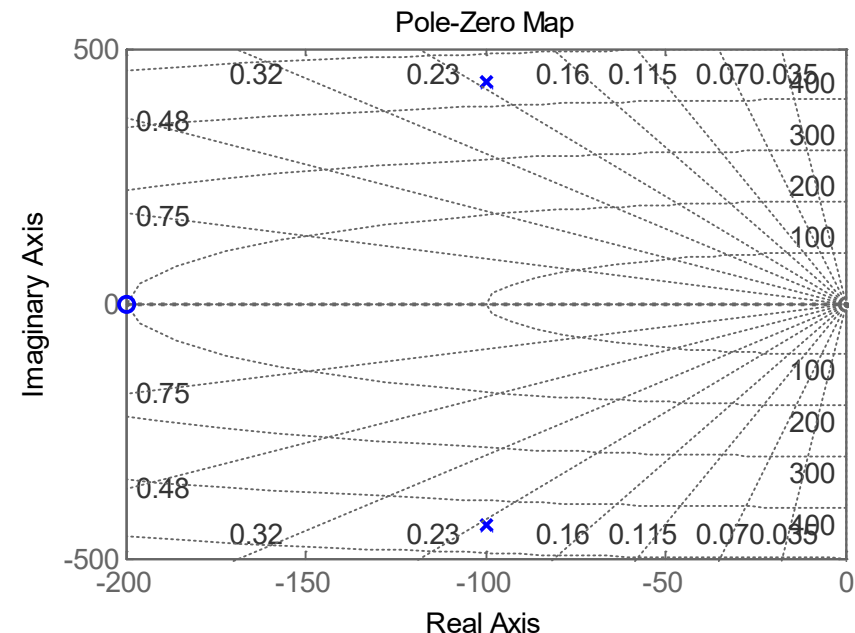
Beispiel: Pol-Nullstellenplan für den R-L-C-Filter

```

      :
%% Zustandsraummodell
zrm= ss(A,B,C,D)

%% Bode-Diagramm
pzmap(zrm);
grid on;

```



4. Dynamische Systeme – 4.2 Plots für dyn. Sys.

`nyquist`-**Befehl**:

Die `nyquist`-Funktion zeichnet die Nyquist-Ortskurve eines oder mehrerer LTI-Systeme:

`nyquist(sys1,...,sysn,w)`

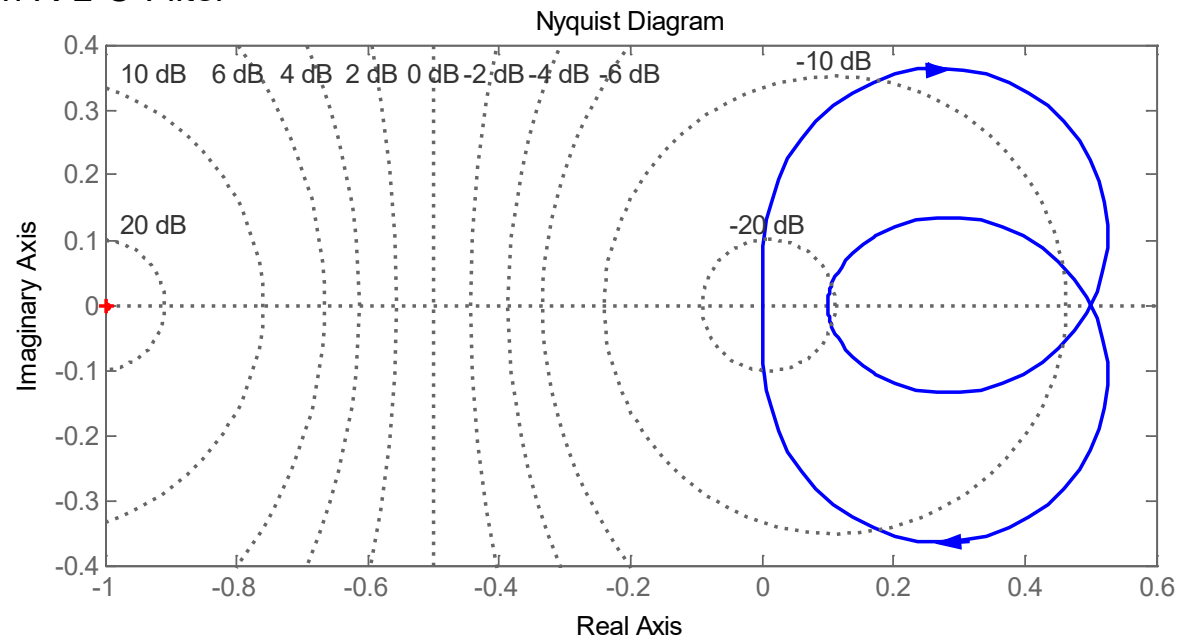
`sys1,...,sysn`: n LTI-Systeme in beliebiger Darstellung (`tf(...)`, `ss(...)`, `zpk(...)`)
`w`: Vektor der den darzustellenden Frequenzbereich definiert.

Beispiel: Nyquist-Ortskurve für den R-L-C-Filter

```

      :
%% Zustandsraummodell
zrm= ss(A,B,C,D)

%% Bode-Diagramm
pzmap(zrm);
grid on;
  
```



4. Dynamische Systeme – 4.2 Plots für dyn. Sys.

step-Befehl:

Die `step`-Funktion zeichnet die Sprungantwort eines oder mehrerer LTI-Systeme auf. Die Höhe des Eingangsprungs beträgt immer den Wert 1:

```
step(sys1,...,sysn,t)
```

`sys1,...,sysn`: n LTI-Systeme in beliebiger Darstellung (`tf(...)`, `ss(...)`, `zpk(...)`)

`t`: Dauer der Sprungantwort

Wird die Zeitdauer t nicht angegeben so wird anhand der Eigenwerte des Systems automatische eine Zeitdauer berechnet.

Für Zustandsraummodelle:

```
[y,t,x]=step(sys,t)
```

`y`: Funktionswerte der Sprungantwort

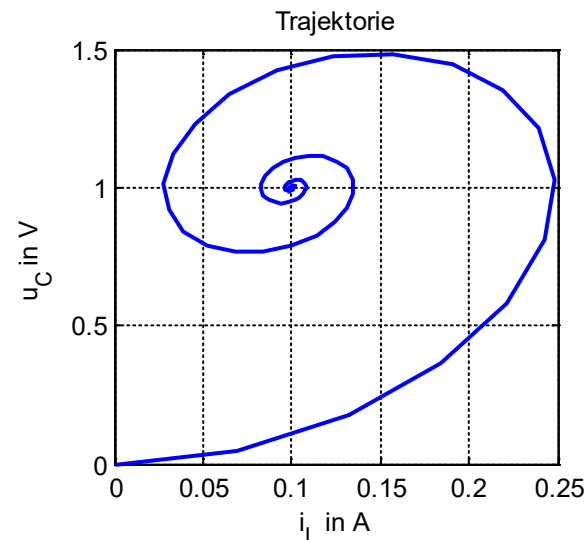
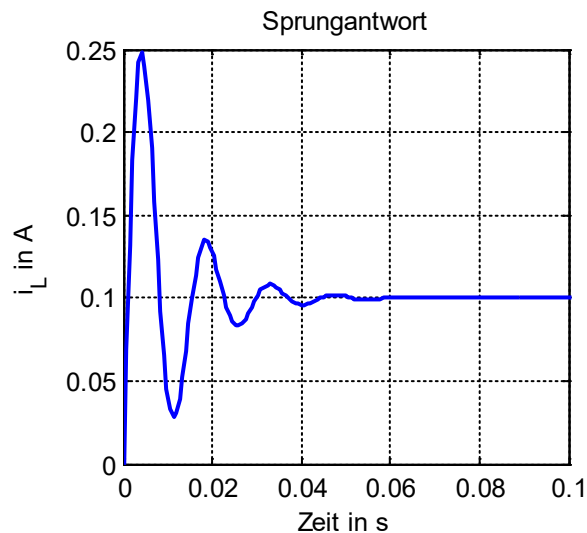
`t`: Zeitwerte der Sprungantwort

`x`: Zustandstrajektorien

4. Dynamische Systeme – 4.2 Plots für dyn. Sys.

Beispiel: Sprungantwort des R-L-C-Filters:

```
⋮  
%% Sprungantwort und Trajektorie plotten  
[y,t,x]=step(zrm,0.1); % Daten für 0.1 s berechnen  
subplot(1,2,1);  
plot(t,y,'linewidth',2.0); % Sprungantwort  
grid on; xlim([0 0.1]);  
subplot(1,2,2);  
plot(x(:,1),x(:,2),'linewidth',2.0); % Trajektorie  
grid on; xlim([0 0.25]);
```



4. Dynamische Systeme – 4.2 Plots für dyn. Sys.

`impulse`-**Befehl**:

Die `impulse`-Funktion zeichnet die Impulsantwort eines oder mehrerer LTI-Systeme auf. Die Höhe des Eingangsimpulses beträgt immer den Wert 1:

```
impulse(sys1,...,sysn,t)
```

`sys1,...,sysn`: n LTI-Systeme in beliebiger Darstellung (`tf(...)`, `ss(...)`, `zpk(...)`)
`t`: Dauer der Impulsantwort

Wird die Zeitdauer t nicht angegeben so wird anhand der Eigenwerte des Systems automatische eine Zeitdauer berechnet.

Für Zustandsraummodelle:

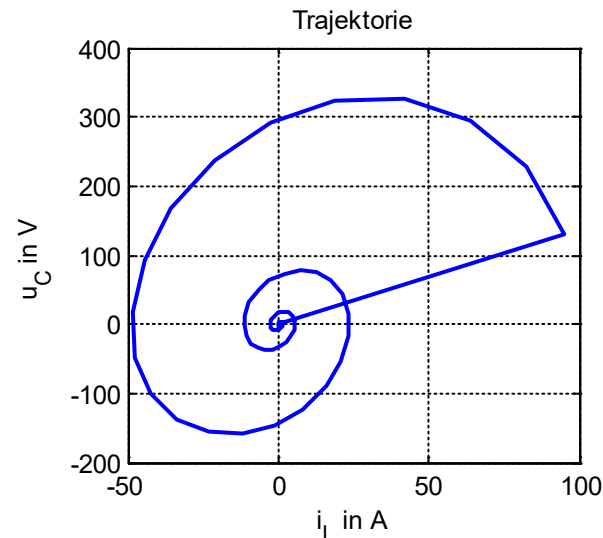
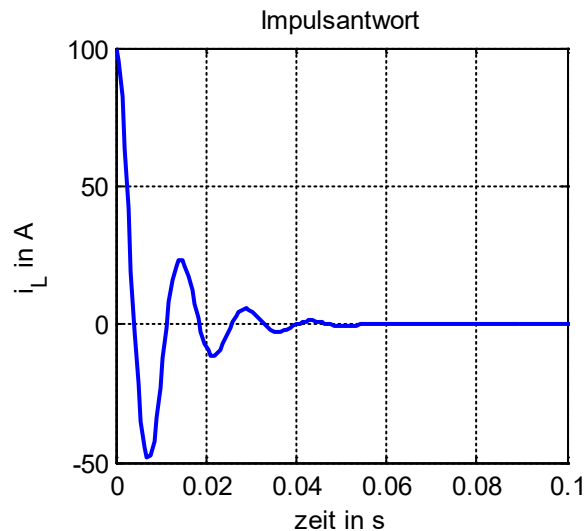
```
[y,t,x]=impulse(sys,t)
```

`y`: Funktionswerte der Sprungantwort
`t`: Zeitwerte der Sprungantwort
`x`: Zustandstrajektorien

4. Dynamische Systeme – 4.2 Plots für dyn. Sys.

Beispiel: Impulsantwort des R-L-C-Filters:

```
⋮  
%% Impulsantwort und Trajektorie plotten  
[y,t,x]=impzrm(0.1); % Daten für 0.1 s berechnen  
subplot(1,2,1);  
plot(t,y,'linewidth',2.0); % Impulsantwort  
grid on; xlim([0 0.1]);  
subplot(1,2,2);  
plot(x(:,1),x(:,2),'linewidth',2.0); % Trajektorie  
grid on;
```



4. Dynamische Systeme – 4.3 Lösung dyn. Sys.

ode##-Befehl:

Mit `odeXX`-Funktion können gewöhnliche Differenzialgleichungen (ODE) mit beliebiger Anregung gelöst werden. Das `##` steht für die von Matlab bereitgestellten Integrationsverfahren. Konkret kann dort `45`, `23`, `113`, `15s`, `23s`, `23t` und `23tb` eingesetzt werden. Eine detailliertere Beschreibung der Integrationsverfahren ist in Kapitel 6 zu finden.

```
[T,Y] = odeXX(odefun,tspan,y0,options)
```

odefun: Function-Handle, welcher Differenzialgleichungen der Form $\dot{y} = f(y, u, t)$ definiert. Auch Differenzialgleichungen mit einer Massenmatrix $M(t, y) \cdot \dot{y} = f(y, u, t)$ können hier definiert werden, dabei muss aber auf die Wahl des richtigen Solvers geachtet werden.

tspan: Zeitintervall über welcher die Differenzialgleichungen gelöst werden soll.

y0: Anfangswerte

options: Struktur, welche die Solver-Einstellungen beinhaltet (siehe `odeset(...)`). Wird dieser Parameter ausgelassen, so werden Standardeinstellungen verwendet.

T: Vektor mit den Zeitpunkten des Lösung.

Y: Vektor mit den Funktionswerten der Lösung.

4. Dynamische Systeme – 4.3 Lösung dyn. Sys.

Function Handle anlegen:

Für ein Function-Handle muss eine separate m-File angelegt werden. In der ersten Zeile wird der Funktionskopf mit Ein- Ausgängen und Funktionsname definiert. Es ist darauf zu achten, dass die m-File den gleichen Namen wie die Funktion aufweist.

```
function v = funName(u)
    :
end
```

Beispiel: Function-Handle für die Differenzialgleichungen des R-L-C-Filter

```
function dx = filterEq(t,x,R,L,C,ue,uet) %uet sind die Zeitwerte von ue
    dx = zeros(2,1); %x1=iL x2=uC
    ue = interp1(uet,ue,t); % Interp. the data set (uit,ui) at time t
    dx(1) = -1/L*x(2)+1/L*ue;
    dx(2) = 1/C*x(1)-1/(R*C)*x(2);
end
```

Die erste Zeile wird benötigt, um einen Vektor mit zwei Einträgen zu definieren. Da die Integrationsverfahren mit variabler Schrittweite arbeiten, muss der Interpolationsbefehl (siehe Kapitel 5.2) verwendet werden um Zwischenwerte für den Eingangsvektor zu bilden. Die Ableitungen werden mit **d** gekennzeichnet.

4. Dynamische Systeme – 4.3 Lösung dyn. Sys.

Function Handle verwenden:

Die angelegte Funktion selbst kann mit dem Operator @ als Objekt übergeben werden.

Beispiel:

```
fun (@(t,x) filterEq(t,x,R,L1,C1,ue,uet))
```

Der Funktion `fun` wird die Funktion `filterEq` übergeben. Die Parameter die bei der Übergabe des Function-Handles nicht festgelegt sind, müssen nach dem @ in Klammern angegeben werden.

4. Dynamische Systeme – 4.3 Lösung dyn. Sys.

odeset-**Befehl:**

Erstellt eine Struktur in der die Optionen der Integrationsverfahren enthalten ist. Dies Struktur kann verwendet werden um die Parameter des Integrationsverfahrens anzupassen.

```
options = odeset('name1',value1,'name2',value2,...)
```

name: Name der Option die geändert werden soll.

value: Parameter der eingestellt werden soll.

Es müssen nur die Optionen angegeben werden die geändert werden müssen. Alle anderen Optionen verbleiben in der Standardeinstellung. Für die Auswahl an Einstellungen siehe in der Matlab Dokumentation unter `odeset`.

Beispiel:

```
options = odeset('RelTol',1e-6,'AbsTol',1e-6);
```

Die oben dargestellten Optionen stellen die Schrittweitenregelung des Integrationsverfahrens ein.

4. Dynamische Systeme – 4.3 Lösung dyn. Sys.

Beispiel: Sprungantwort des R-L-C-Filter:

```
%% Parameter des RLC Filters
L=10e-3;
C=500e-6;
R=10;
ue = 24;

% Anregung (Sprungfunktion)
uet = linspace(0,12,1000); % Zeitpunkte von t=0 bis t=12 verteilt.
ue = 1.*(uet>0.1); % Sprungfunktion bei t = 0.1

% ODE45 Solver
options = odeset('RelTol',1e-6,'AbsTol',1e-6);
[T,Y]=ode45(@(t,x) filterEq(t,x,R,L,C,ue,uet),[0 0.3],[0 0],options);
figure();
plot(T,Y(:,1))
```

```
function dx = filterEq(t,x,R,L,C,ue,uet)
    dx = zeros(2,1); %x1=iL x2=uC
    ue = interp1(uet,ue,t); % Interp. the data set (uit,ui) at time t
    dx(1) = -1/L*x(2)+1/L*ue;
    dx(2) = 1/C*x(1)-1/(R*C)*x(2);
end
```

filterEq.m

4. Dynamische Systeme – 4.3 Lösung dyn. Sys.

Beispiel: Sprungantwort des R-L-C-Filter:

